

CF Bulk Upload — Step-by-Step with Code and a Worked Example

This is the developer reference for the bulk upload engine. It follows one sheet — `offerBulkUpload.xlsx` — from the moment it lands in the DAM to the moment the report is written, showing the code shape and the data at every step.

Package names, model paths, and element names are illustrative — swap in the real ones from `/conf/aemcloudproject/settings/dam/cfm/models`.

The sample data we trace throughout:

OfferDetail	row 4:	offerSlug=duty-free-shopping
		categories=[travel]
		merchants=[shilla-duty-free, shinsegae-duty-free, dufry-thomas-julie]
		cards=[visa] ctas=[register-now, redeem-now]
MerchantDetails	row 2:	merchantSlug=shilla-duty-free
		venues=[shilla-duty-free-macau, shilla-duty-free-hongkong]
merchantVenues	row 2:	venueSlug=shilla-duty-free (only this one exists)
OfferCategory	row 2:	categorySlug=health-fitness parent=restrelax

Four references in that data are broken (`dufry-thomas-julie`, `register-now`, the two `shilla-*` venues, `restrelax`). That is deliberate — it's what a hand-made sheet really looks like, and it's how we show the failure paths, not just the happy path.

Step 0 — The one rule everything follows

A Content Fragment reference is stored in the JCR as a **plain string path**. The repository does not check it. Write a reference to a fragment that doesn't exist and AEM accepts it silently; the broken link only surfaces when an author opens the offer weeks later and a section renders empty.

So the whole engine is built to guarantee: **never write a reference until the fragment it points to has a real, known path**. Every design choice below serves that rule.

Step 1 — Upload lands, the workflow step wakes up

The author drops the file into `/content/dam/aemcloudproject/bulk-upload/inbox/`. A Workflow Launcher on that path fires a one-step workflow. The step is deliberately thin — it unwraps the payload and hands a stream to the engine. No business logic lives here.

--	--	--


```

@Component(service = WorkflowProcess.class,
    property = {"process.label=CF Bulk Upload"})
public class BulkUploadProcess implements WorkflowProcess {

    @Reference private BulkUploadService engine;
    @Reference private ResourceResolverFactory rrf;

    @Override
    public void execute(WorkItem item, WorkflowSession wfSession, MetaDataMap args)
        throws WorkflowException {

        String runId = LocalDateTime.now().format(RUN_ID_FMT) + "-" + shortRandom();
        MDC.put("bulkRunId", runId);    // every log line now carries the run id

        String assetPath = item.getWorkflowData().getPayload().toString();

        Map<String, Object> auth =
            Map.of(ResourceResolverFactory.SUBSERVICE, "bulk-upload-writer");

        try (ResourceResolver resolver = rrf.getServiceResourceResolver(auth)) {
            Asset asset = resolver.getResource(assetPath).adaptTo(Asset.class);

            // Gotcha: on AEMaaCS the node can exist before its binary is readable.
            InputStream xlsx = asset.getOriginal().getStream();
            if (xlsx == null) {
                throw new WorkflowException("Binary not ready for " + assetPath);
            }

            BulkOptions options = BulkOptions.builder()
                .runId(runId)
                .dryRun(false)
                .multifieldMode(MultifieldMode.REPLACE)
                .referencePolicy(ReferencePolicy.STRICT)
                .build();

            BulkReport report = engine.run(xlsx, resolver, options);

            item.getWorkflow().getWorkflowData().getMetaDataMap()
                .put("reportPath", report.getPath());

        } catch (Exception e) {
            // Let the step fail so the workflow queue retries it. The engine is
            // idempotent, so a retry is safe.
            throw new WorkflowException("Bulk upload failed, run " + runId, e);
        }
    }
}

```

```
} finally {  
    MDC.remove("bulkRunId");  
}  
}  
}
```

Why a service user, not the workflow's session: the service user's ACLs (set up once via repointit) are what physically stop this code from writing outside the CF tree, the sequence node, and the reports folder. It is a guardrail, not a formality.

Everything below is inside `engine.run(...)`. The engine took a stream and a resolver. It has no idea a workflow exists — which is exactly why you can unit-test it with a plain `FileInputStream`.

Step 2 — Parse: Excel becomes typed Java objects

Plain-language: the workbook is really six database tables. This step loads each sheet into a list of objects, and nothing touches the repository yet.

Two rules make this robust:

Find columns by header name, not by position — so a reordered column doesn't silently shift your data, and a *missing required* column aborts the run with one clear message instead of 500 downstream errors.

Handle "spill-down" — a row with a slug starts a record; blank-slug rows beneath it add their cell values to that record's lists, each column independently.



java

```

public List<OfferDetailRow> parseOfferDetail(Sheet sheet) {
    Map<String,Integer> col = headerIndex(sheet);    // "offerSlug" -> 2, ...
    require(col, "offerSlug", "offerCategories", "offerMerchants",
            "offerCards", "offerCta");    // missing header -> abort run

    List<OfferDetailRow> out = new ArrayList<>();
    OfferDetailRow current = null;

    for (int r = 1; r <= sheet.getLastRowNum(); r++) {    // r=0 is the header
        Row row = sheet.getRow(r);
        String slug = trimLower(cell(row, col.get("offerSlug")));

        if (!slug.isEmpty()) {    // a new record starts
            current = new OfferDetailRow();
            current.sheetRow = r + 1;    // 1-based, for the report
            current.offerSlug = slug;
            current.offerId = cell(row, col.get("offerID")); // may be blank
            out.add(current);
        }
        if (current == null) continue;    // stray row before any slug

        // spill-down: add this row's values to the open record, per column
        addIfPresent(current.categories, cell(row, col.get("offerCategories")));
        addIfPresent(current.merchants, cell(row, col.get("offerMerchants")));
        addIfPresent(current.cards, cell(row, col.get("offerCards")));
        addIfPresent(current.ctas, cell(row, col.get("offerCta")));
    }
    return out;
}

private void addIfPresent(List<String> list, String v) {
    v = trimLower(v);
    if (!v.isEmpty()) list.add(v);
}

```

What comes out for our sample — three physical rows collapse into one object:

□
java

```
OfferDetailRow {
  sheetRow = 4,
  offerSlug = "duty-free-shopping",
  offerId = "811806",
  categories= ["travel"],
  merchants = ["shilla-duty-free","shinsegae-duty-free","dufry-thomas-julie"],
  cards = ["visa"],
  ctas = ["register-now","redeem-now"]
}
```

Note `sheetRow = 4` is carried forever — every message in the final report points the author at a real line in their file.

POI in OSGi: wrap `new XSSFWorkbook(stream)` in the thread-context-classloader swap proven in spike 0.1, or this throws `NoClassDefFoundError` at runtime.

Step 3 — Validate: judge the data before touching anything

Plain-language: check the whole workbook for problems while it's still just objects in memory. Cheap to do, and it means a bad file is rejected before we've created a single fragment. Each problem becomes a structured issue that carries its sheet, row, and reason.


java

```

public List<Issue> validate(Workbook wb, ParsedData data) {
    List<Issue> issues = new ArrayList<>();

    // 3a. field-level checks
    for (OfferDetailRow o : data.offers) {
        if (isBlank(o.offerSlug))
            issues.add(Issue.error("OfferDetail", o.sheetRow, "offerSlug", "missing slug"));
        if (!o.offerSlug.matches("^[a-z0-9][a-z0-9-]*$"))
            issues.add(Issue.error("OfferDetail", o.sheetRow, "offerSlug",
                                    "invalid slug: " + o.offerSlug));
    }

    // 3b. duplicate keys within a sheet
    flagDuplicates(data.offers, o -> o.offerSlug, "OfferDetail", issues);

    // 3c. foreign keys: every referenced slug must resolve somewhere
    Set<String> venuesInSheet = slugSet(data.venues);
    for (MerchantRow m : data.merchants) {
        for (String v : m.venues) {
            if (!venuesInSheet.contains(v) && !existsInDam(VENUES, v)) {
                issues.add(Issue.error("MerchantDetails", m.sheetRow, "venues",
                                        "venue not found in sheet or DAM: " + v));
            }
        }
    }

    // ... same pattern for offer.categories / merchants / cards / ctas

    // 3d. category parent cycles (parent points at another category)
    detectCycles(data.categories, issues);

    return issues;
}

```

What it finds in our sample:

```

❑
ERROR MerchantDetails row 2 venues venue not found: shilla-duty-free-macau
ERROR MerchantDetails row 2 venues venue not found: shilla-duty-free-hongkong
ERROR OfferDetail row 4 merchants merchant not found: dufry-thomas-julie
ERROR OfferDetail row 4 ctas cta not found: register-now
WARN OfferCategory row 2 parent parent not found: restrelax (reference left empty)

```

The dry-run fork lives right here. If `options.dryRun` is true, we stop after this step and report what *would* happen. Same parse, same validation, zero writes — which is why the dry-run result is trustworthy: it can't disagree with the real run because it is the real run minus the write phase.

Step 4 — Resolve: decide CREATE vs UPDATE for each record

Plain-language: for every record, look up whether its fragment already exists, and remember the answer in a map. Because the slug is the node name, "does it exist?" is a single direct path lookup — no repository-wide query, no custom index on the hot path.

```
□
java
public Outcome resolve(String slug, String folder, ResourceResolver resolver) {
    String path = basePath + "/" + folder + "/" + JcrUtil.createValidName(slug);
    Resource existing = resolver.getResource(path);

    if (existing == null) {
        return Outcome.toCreate(slug, path);          // will create
    }
    ContentFragment cf = existing.adaptTo(ContentFragment.class);
    if (cf == null) {
        return Outcome.error(slug, path, "path exists but is not a Content Fragment");
    }
    return Outcome.toUpdate(slug, path, cf);          // will update
}
```

Run over every slug, the results go into the **outcome map** — the single source of truth the write phase reads paths from:

```
□
java
Map<String, Outcome> outcomes = new HashMap<>();
// "duty-free-shopping" -> CREATE at .../offer-listing/duty-free-shopping
// "shilla-duty-free"   -> CREATE at .../merchants/shilla-duty-free
// "travel"             -> CREATE at .../categories/travel
```

Why slug-as-node-name matters here: the alternative — storing the slug as a property and finding fragments with a JCR query on `@offerSlug` — needs a custom Oak index, gets slower as content grows, and can return stale results mid-run. A path lookup is $O(1)$, index-free, and always current.

Step 5 — Write, from the bottom of the dependency graph up

Plain-language: we write the things that depend on nothing first, commit them, then write the things that depend on *those*, and so on. By the time we write an offer, every fragment it points to already exists at a path we can read from the outcome map — so no reference is ever a guess.

The order is **computed** from who-references-whom, not hardcoded:



Tier 1 (reference nothing): venues, cards, ctas, categories(scalar fields)

Tier 1b: categories(parent wiring, 2nd pass)

Tier 2 (need tier 1): merchants -> reference venues

Tier 3 (need everything): offers -> reference categories/merchants/cards/ctas

5a — The create primitive (same everywhere)



java

```
private Outcome createFragment(FragmentTemplate template, Resource parentFolder,
    String slug, Map<String,Object> elements) {
    String nodeName = JcrUtil.createValidName(slug);
    ContentFragment cf = template.createFragment(parentFolder, nodeName, slug);

    for (var e : elements.entrySet()) {
        cf.getElement(e.getKey()).setValue(toFragmentData(e.getValue()));
    }
    return Outcome.created(slug, cf.adaptTo(Resource.class).getPath());
}
```

5b — Tier 1: venues (no dependencies)



java

```
for (VenueRow v : data.venues) {
    Outcome o = outcomes.get(v.venueSlug);
    if (o.action == CREATE) {
        createFragment(venueTemplate, venuesFolder, v.venueSlug, Map.of(
            "venueName", v.name,
            "address", v.address));
        o.status = CREATED;
    }
    commitEvery(50, resolver);
}
resolver.commit(); // hard commit at the tier boundary
```

Result for our sample:



CREATED .../merchants/venues/shilla-duty-free

5c — Tier 1: categories in two passes

Pass 1 writes the scalar fields and **skips** `parent`, because the parent category may not exist yet. Pass 2 wires parents once every category has a path.

```

☐
java
// pass 1 — create, no parent
for (CategoryRow c : data.categories) {
    createFragment(categoryTemplate, categoriesFolder, c.categorySlug, Map.of(
        "categoryName", c.name));
    outcomes.get(c.categorySlug).status = CREATED;
}
resolver.commit();

// pass 2 — wire parent references now that all paths exist
for (CategoryRow c : data.categories) {
    if (isBlank(c.parent)) continue;
    Outcome parent = outcomes.get(c.parent);
    if (parent == null || parent.status == FAILED) {
        report.warn("OfferCategory", c.sheetRow,
            "parent not found: " + c.parent + " — left empty");
        continue;           // don't write a broken reference
    }
    ContentFragment cf = outcomes.get(c.categorySlug).fragment;
    cf.getElement("parent").setValue(FragmentData.of(parent.path));
}
resolver.commit();
```

For `health-fitness`, parent `restrelax` resolves to nothing → we leave the reference empty and record a warning, rather than writing a path that points nowhere. **Two passes plus cycle detection (Step 3d) are what make self-referential categories safe.**

5d — Tier 2: merchants (reference venues)

Now the payoff. A merchant's `venues` element is built by reading **paths out of the outcome map** — never by string-building a guess:

```

☐
java
```

```

for (MerchantRow m : data.merchants) {
    List<String> venuePaths = new ArrayList<>();
    boolean broken = false;

    for (String vSlug : m.venues) {
        Outcome vo = outcomes.get(vSlug);
        if (vo == null || vo.status == FAILED) {
            report.fail("MerchantDetails", m.sheetRow,
                "venue unresolved: " + vSlug);
            broken = true;
        } else {
            venuePaths.add(vo.path);          // a real, committed path
        }
    }

    if (broken && options.referencePolicy == STRICT) {
        outcomes.get(m.merchantSlug).status = FAILED;
        continue;                          // skip this merchant, keep going
    }

    createFragment(merchantTemplate, merchantsFolder, m.merchantSlug, Map.of(
        "merchantName", m.name,
        "venues", venuePaths.toArray(String[]::new));
    outcomes.get(m.merchantSlug).status = CREATED;
}
resolver.commit();

```

For our sample, `shilla-duty-free`'s two venues were never created (they weren't in the venues sheet), so under STRICT policy:

☐

FAILED MerchantDetails row 2 shilla-duty-free (venues macau + hongkong unresolved)

Its failure is now recorded in the outcome map — which the next tier will see.

5e — Tier 3: offers (reference everything)

Each offer checks the recorded outcome of every slug it points at. Notice how the `shilla-duty-free` failure from the tier below **cascades** up automatically:

☐

java

```

for (OfferDetailRow off : data.offers) {
    List<String> merchantPaths = resolveRefs(off.merchants, outcomes, report,
        "OfferDetail", off.sheetRow, "merchants");
    List<String> ctaPaths    = resolveRefs(off.ctas, outcomes, report,
        "OfferDetail", off.sheetRow, "ctas");
    // ... categories, cards

    if (report.hasFailureFor(off.sheetRow) && options.referencePolicy == STRICT) {
        outcomes.get(off.offerSlug).status = FAILED;
        continue;
    }

    Outcome o = outcomes.get(off.offerSlug);
    if (o.action == CREATE) {
        String offerId = isBlank(off.offerId) ? idService.next("offer") : off.offerId;
        createFragment(offerTemplate, offerFolder, off.offerSlug, Map.of(
            "offerId", offerId,
            "categories", categoryPaths.toArray(String[]::new),
            "merchants", merchantPaths.toArray(String[]::new),
            "cards", cardPaths.toArray(String[]::new),
            "ctas", ctaPaths.toArray(String[]::new)));
        o.status = CREATED;
    } else {
        updateFragment(o, off);    // see Step 6
    }
}
resolver.commit();

```

`duty-free-shopping` fails for three independent reasons, one of them a cascade:

❑

FAILED OfferDetail row 4 duty-free-shopping

- merchant dufry-thomas-julie not found
- merchant shilla-duty-free failed at row 2 <-- cascaded from tier 2
- cta register-now not found

That cascade being *visible* — rather than producing an offer that points at a merchant that was never created — is the entire reason for going bottom-up.

Step 6 — Update an existing fragment (diff, version, write only changes)

Plain-language: on a re-run, most fragments already exist. We compare the sheet's values to what's

stored. If nothing changed, we write nothing and record `UNCHANGED`. If something changed, we snapshot a version first (so the run is reversible) and write only the fields that differ.

```

❏
java
private void updateFragment(Outcome o, OfferDetailRow off) {
    ContentFragment cf = o.fragment;
    Map<String,Object> changes = diff(cf, off);    // only fields that differ

    if (changes.isEmpty()) {
        o.status = UNCHANGED;                // no version, no write
        return;
    }
    // snapshot BEFORE modifying, tagged with the run id, so we can roll back
    cf.adaptTo(Resource.class).getResourceResolver()
        .adaptTo(Session.class).getWorkspace().getVersionManager()
        .checkpoint(cf.adaptTo(Resource.class).getPath());

    for (var e : changes.entrySet()) {
        cf.getElement(e.getKey()).setValue(toFragmentData(e.getValue()));
    }
    o.status = UPDATED;
}

```

The `diff` is what stops a re-run of an unchanged sheet from stamping a pointless new version on all 500 fragments and reporting "500 updated" when nothing actually changed. Run the same sheet twice and the second run reports `UNCHANGED` across the board — which is your cheapest proof that the whole pipeline is idempotent.

Multifield replace vs merge lives in `diff`. Under REPLACE (recommended), sheet `[A,D]` over stored `[A,B,C]` yields `[A,D]`; under MERGE it yields `[A,B,C,D]`. Replace makes the sheet the source of truth; merge makes it impossible to ever remove a reference. This is a business decision — expose it as config, don't bury it in code.

Step 7 — Batched commits and failure isolation

Plain-language: don't save after every single fragment (slow) and don't save only once at the very end (one bad row loses everything). Save in batches, and when a batch fails, undo just that batch and carry on.

```

❏
java

```

```
private int pending = 0;
private void commitEvery(int n, ResourceResolver resolver) {
    if (++pending >= n) {
        try {
            resolver.commit();
            resolver.refresh();
        } catch (PersistenceException e) {
            resolver.revert();           // drop this batch's changes
            report.failBatch(e);         // mark them FAILED, keep the run alive
        }
        pending = 0;
    }
}
```

One malformed row reverts its batch of 50 and the run continues; it never takes down the other 450.

Step 8 — ID generation that is safe across AEMaaCS pods

Plain-language: when the sheet leaves an ID blank, we generate one. AEMaaCS runs multiple author pods, so a naive “read counter, add one, write it back” will hand the same number to two pods at once. We use optimistic retry: if someone else bumped the counter while we weren’t looking, we re-read and try again.

```

❑
java
public String next(String type) {
    for (int attempt = 0; attempt < 5; attempt++) {
        try {
            Resource seq = resolver.getResource("/var/aemcloudproject/sequences/" + type);
            ModifiableValueMap vm = seq.adaptTo(ModifiableValueMap.class);
            long current = vm.get("value", 0L);
            vm.put("value", current + 1);
            resolver.commit();           // fails if another pod committed first
            return type + "-" + (current + 1);
        } catch (PersistenceException retry) {
            resolver.refresh();         // re-read the latest value, loop
        }
    }
    throw new IllegalStateException("Could not allocate id for " + type);
}
```

The generated `offerId` is written once at creation and never changed on update — it’s a stable identifier, unlike the slug which the author owns.

Step 9 — Report: turn the recorded outcomes into a file

Plain-language: every step above appended to an in-memory log as it went. Now we write that log out as an Excel file the author can open — a summary sheet with counts, and a details sheet with one row per fragment and what happened to it.

```

□
java
public String writeReport(BulkReport report, ResourceResolver resolver, BulkOptions opt) {
    try (XSSFWorkbook wb = new XSSFWorkbook()) {
        Sheet summary = wb.createSheet("Summary");
        writeRow(summary, "Run id", opt.runId);
        writeRow(summary, "Created", report.count(CREATED));
        writeRow(summary, "Updated", report.count(UPDATED));
        writeRow(summary, "Unchanged", report.count(UNCHANGED));
        writeRow(summary, "Failed", report.count(FAILED));

        Sheet details = wb.createSheet("Details");
        header(details, "sheet", "row", "slug", "type", "action", "cfPath", "message");
        for (ReportLine l : report.lines()) {
            writeRow(details, l.sheet, l.row, l.slug, l.type,
                l.action, l.cfPath, l.message);
        }

        String path = basePath + "../reports/" + opt.runId + "_report.xlsx";
        // stream wb into a new DAM asset at `path` (AssetManager.createAsset)
        return path;
    }
}
```

The details sheet for our sample run:

sheet	row	slug	type	action	cfPath	message
merchantVenues	2	shilla-duty-free	venue	CREATED	.../venues/shilla-duty-free	
OfferCategory	2	health-fitness	category	CREATED	.../categories/health-fitness	parent restrelax not found, left empty
OfferCategory	3	travel	category	CREATED	.../categories/travel	
offerCard	2	visa	card	CREATED	.../cards/visa	
MerchantDetails	2	shilla-duty-free	merchant	FAILED		venues macau + hongkong not found
OfferDetail	4	duty-free-shopping	offer	FAILED		dufry-thomas-julie missing; shilla-duty-free failed row 2; register-now missing

The author reads exactly which rows failed and why, fixes those cells, and re-uploads. Because the run is idempotent, the fragments that already succeeded come back `UNCHANGED` on the second pass — only the fixed rows do new work.

Step 10 — Finish and clean up

Write `reportPath` into the workflow metadata so it shows in the author's Timeline.

Move the source file from `inbox/` to `processed/` (or `failed/`) — keeps the inbox clean and, because those folders don't match the launcher path, moving the file doesn't re-fire the workflow.

Optionally email the initiator the report link.

The whole thing in one breath

Upload → launcher fires a workflow → thin step hands the stream to the engine → parse to objects → validate as a graph (dry-run stops here) → resolve create-vs-update per slug → write bottom-up so every reference resolves to a committed path → diff-and-version on updates → report every outcome to an xlsx → move the file and finish. Every rule exists to keep two promises: **run it twice, get the same result**, and **when rows fail, the author can see exactly which and why**.